Amazon VPC & VPC Peering

Fundamentals • Purpose • Why we use it • Simple examples • Troubleshooting

A reusable concept-refresh document based on your VPC Peering project, with technical points checked against current AWS documentation.

Designed for quick revision whenever you forget a concept.

1. The Big Picture

VPC: An Amazon VPC is your logically isolated virtual network in AWS. You control its IP ranges, subnets, routing, and network access.

Purpose: AWS resources need a network in which they can communicate while you control how that communication happens.

Easy analogy: VPC = a private office building. Subnets = different areas. Route tables = directions. Security groups = doors/security guards.

Your project: You created two VPCs and connected them with VPC Peering so EC2 resources could communicate using private IP addresses.

Core flow: VPC → Subnet → EC2 → Route Table → VPC Peering → Route Table → Security Group → EC2

2. CIDR Blocks

What? CIDR describes an IP address range. Example: **10.0.0.0/16**.

Why? Every VPC needs an IP range from which private addresses can be assigned to subnets and resources.

Example: VPC A = 10.0.0.0/16. VPC B = 10.1.0.0/16. An address such as 10.0.1.10 belongs to A, while 10.1.1.10 belongs to B.

Peering rule: The VPCs must not have overlapping IPv4 CIDR blocks. AWS does not establish peering between overlapping IPv4 ranges.

Project note: Your text says VPC 1 = 10.0.0.0/16 and VPC 2 = 10.1.0.0/16, but later mentions a 10.2.0.0/16 route. If VPC 2 is really 10.1.0.0/16, the peer route should target 10.1.0.0/16. Always use the actual peer VPC CIDR.

3. Subnets

What? A subnet is a smaller IP range inside a VPC where resources such as EC2 instances are placed.

Example: VPC = 10.0.0.0/16; subnet = 10.0.1.0/24; EC2 could receive 10.0.1.25.

Public subnet: Its route table has a path toward an Internet Gateway. A resource may also need a public IPv4 address for internet-facing access.

Private subnet: Commonly used for internal resources that should not be directly reachable from the public internet.

For peering: The key requirement is that the subnet's route table contains the route needed to reach the peer VPC.

4. Route Tables

What? A route table contains rules that tell AWS where network traffic should go.

Easy analogy: It is like GPS: “For this destination, take this road.”

For peering: VPC A needs a route to VPC B's CIDR, targeting the peering connection. VPC B needs the reverse route.

VPC	Destination	Target
A	10.0.0.0/16 (local)	local
A	10.1.0.0/16 (peer)	VPC peering connection
B	10.1.0.0/16 (local)	local
B	10.0.0.0/16 (peer)	VPC peering connection

Memory: A route tells traffic how to get there; the return route tells the response how to get back.

5. VPC Peering

What? A private network connection between two VPCs.

Why? It lets resources in separate VPCs communicate using private addresses without routing the peering traffic through the public internet.

Example: VPC A contains an application server; VPC B contains a database. Peering can let the application reach the database's private IP.

Requester: Creates/initiates the peering request.

Accepter: Accepts the request. The connection becomes active after acceptance.

Important: Peering does not automatically create the routes or security permissions you need.

Non-transitive: If A↔B and A↔C exist, B cannot use A as a transit path to C. Direct peering between B and C would be needed.

6. Security Groups

What? A virtual firewall associated with resources such as EC2 instances.

Purpose: Controls which inbound and outbound traffic is allowed.

Why did ping fail? Ping uses ICMP. If the destination security group does not allow the required ICMP traffic, ping can fail even when the peering and routes are correct.

Simple idea: The route table says “the packet can reach VPC B,” but the security group can still say “do not allow this traffic.”

7. Network ACLs (NACLs)

What? A subnet-level traffic filter.

Why? It adds another layer of network access control in addition to security groups.

Easy difference: Security Group → resource-level firewall. NACL → subnet-level filter.

Peering troubleshooting: If routes and security groups look correct but traffic still fails, check the NACLs too.

8. EC2 + Private/Public IP + Elastic IP

Private IP: Internal address used for communication inside AWS private networking, including VPC peering.

Public IPv4: Address used for internet-facing connectivity when the surrounding network configuration allows it.

Elastic IP: A static public IPv4 address that can be associated with an EC2 instance.

Important: An Elastic IP is **not required for VPC Peering**. Your project used it because the EC2 Instance Connect access path required a public address.

Modern alternative: AWS also documents EC2 Instance Connect Endpoint for connecting to instances without requiring a public IPv4/IPv6 address.

9. EC2 Instance Connect

What? A way to access an EC2 instance, including from the AWS console, without relying on manually managed long-lived SSH private keys in the same way as traditional SSH access.

Why in this project? You needed to log into Instance 1 and run a connectivity test against Instance 2.

Do not confuse: EC2 Instance Connect answers “How do I access the server?” VPC Peering answers “How do two VPC networks communicate?”

10. Ping & ICMP

Ping: A simple network reachability test using ICMP Echo Request/Reply.

Example: If Instance 1 = 10.0.1.10 and Instance 2 = 10.1.1.10, run **ping 10.1.1.10** from Instance 1.

If it succeeds: The tested network path and required ICMP permissions are working.

If it fails: Check the peer status, both route tables, security groups, NACLs, instance state, and the destination private IP.

11. Complete Mental Model

AWS concept	Easy analogy	Main job
VPC	Building	Private network boundary
Subnet	Floor/section	Smaller IP area for resources
EC2	Server/computer	Runs workloads
Private IP	Internal address	Internal communication
Route table	GPS/directions	Chooses where traffic goes
VPC peering	Private road	Connects two VPCs
Security group	Door/security guard	Allows/blocks resource traffic
NACL	Checkpoint	Filters subnet traffic
EC2 Instance Connect	Access method	Lets you access the server
Elastic IP	Stable public address	Provides static public IPv4

One-line flow: EC2 in VPC A wants VPC B → route table sends peer traffic to the peering connection → VPC B has a return route → security/NACL rules permit the traffic → communication works.

12. Troubleshooting Checklist

When two EC2 instances in peered VPCs cannot communicate, check in this order:

• **1. CIDRs:** Confirm the VPC ranges do not overlap.

• **2. Peering:** Confirm the peering connection is active.

• **3. VPC A route:** Route VPC B's CIDR to the peering connection.

• **4. VPC B route:** Route VPC A's CIDR to the peering connection.

• **5. Security groups:** Allow the required protocol/port from the source.

• **6. NACLs:** Confirm subnet-level rules allow the traffic.

• **7. Correct address:** Use the destination instance's private IP for the peering test.

• **8. Instance access:** If EC2 Instance Connect itself fails, troubleshoot its access requirements separately.

• **9. Reachability Analyzer:** Use it to help identify whether routing, security groups, or NACLs are causing the failure.

13. Exam / Interview Quick Revision

VPC: Isolated virtual network in AWS.

Subnet: Smaller IP range inside a VPC.

CIDR: Defines an IP address range.

Route table: Decides where traffic goes.

VPC Peering: Private connection between two VPCs.

Requester: Initiates the peering request.

Accepter: Accepts the request.

Key requirement: VPC CIDRs must not overlap.

Key routing requirement: Both sides need routes to the peer CIDR.

Security Group: Controls allowed traffic for resources such as EC2.

NACL: Filters traffic at the subnet level.

Ping: Uses ICMP to test reachability.

Elastic IP: Static public IPv4; not required for peering itself.

Transitive peering: Not supported.

Best memory sentence: “Peering creates the connection; route tables tell traffic to use it; security controls decide whether traffic is allowed.”

14. What This Project Really Taught You

The most important lesson is that **network connectivity is a chain**. Creating a peering connection alone is not enough.

For Instance 1 in VPC A to reach Instance 2 in VPC B:

• The VPC address ranges must be non-overlapping.

• The peering connection must be active.

• VPC A needs a route toward VPC B.

• VPC B needs a return route toward VPC A.

• The destination security group must permit the required traffic.

• The relevant NACLs must permit the traffic.

• The test must use the correct destination address and protocol.

Once you understand this chain, VPC connectivity troubleshooting becomes a process of finding which link is missing.

Verification sources: AWS official VPC Peering, Route Tables, EC2 Instance Connect, EC2 IP Addressing, and VPC Peering Troubleshooting documentation. Source links are provided below for verification.

AWS VPC Peering Basics: https://docs.aws.amazon.com/vpc/latest/peering/vpc-peering-basics.html

AWS Peering Routing: https://docs.aws.amazon.com/vpc/latest/peering/vpc-peering-routing.html

AWS Peering Creation: https://docs.aws.amazon.com/vpc/latest/peering/create-vpc-peering-connection.html

AWS EC2 Instance Connect Prerequisites: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-instance-connect-prerequisites.html

AWS EC2 IP Addressing: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/using-instance-addressing.html

AWS Peering Troubleshooting: https://docs.aws.amazon.com/vpc/latest/peering/troubleshoot-vpc-peering-connections.html